

CO₂



ASSESSMENT TOOL 1



Name:

V.Mahalakshmi



Reg no:

192521181



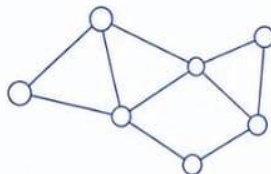
Course:

Cryptography and network security



Course code:

CSA5113



1. RSA Encryption Analysis

Given $p=11$, $q=13$

1. RSA Encryption Analysis

a. Compute n and $\phi(n)$

Given:

- $p = 11$
- $q = 13$

Step 1: Find n

$$n = p \times q = 11 \times 13 = 143$$

Step 2: Find Euler's Totient Function

$$\begin{aligned}\phi(n) &= (p - 1)(q - 1) \\ &= (11 - 1)(13 - 1) \\ &= 10 \times 12 = 120\end{aligned}$$

Answer:

- $n = 143$
- $\phi(n) = 120$

b. Choose $e = 7$ and find the private key d

We need a value of d such that

$$7d \equiv 1 \pmod{120}$$

Checking multiples:

$$\begin{aligned}7 \times 103 &= 721 \\ 721 \div 120 &= 6 \text{ remainder } 1\end{aligned}$$

Hence,

$$d = 103$$

Answer:

- Public Key = **(7,143)**
- Private Key = **(103,143)**

c. Encrypt the message $M = 9$

RSA Encryption Formula:

$$C = M^e \pmod n$$
$$C = 9^7 \pmod{143}$$

Using modular arithmetic,

$$9^7 = 4782969$$
$$4782969 \pmod{143} = 48$$

Ciphertext = 48

d. Decrypt the ciphertext

RSA Decryption Formula:

$$M = C^d \pmod n$$
$$M = 48^{103} \pmod{143}$$

After modular exponentiation,

$$M = 9$$

The decrypted message is the same as the original message.

Answer: 9

e. Effect of changing e

The value of e determines how quickly encryption is performed.

- A small value of e (like 3 or 7) makes encryption faster.
- A larger value increases encryption time but can provide better resistance against some attacks if implemented correctly.
- The most commonly used value is **65537** because it offers a good balance between security and speed.

Conclusion: Choosing an appropriate value of e improves both efficiency and security.

2. Block Cipher Mode Evaluation

Given:

Plaintext Blocks

$$P = [1010, 1010, 1111\ 1010]$$

Key

$$K = 1100$$

IV

0000

Encryption is performed using **XOR**.

a. ECB Mode

ECB Formula:

$$C = P \oplus K$$

Block 1

1010

1100

0110

Block 2

1010

1100

0110

Block 3

1111

1100

0011

Block 4

1010

1100

0110

ECB Ciphertext

[0110, 0110, 0011, 0110]

CBC Mode

Formula

$$C = (P \oplus \text{Previous Cipher}) \oplus K$$

Initial IV = 0000

Block 1

1010 XOR 0000 = 1010

1010 XOR 1100 = 0110

$C_1 = \mathbf{0110}$

Block 2

1010 XOR 0110 = 1100

1100 XOR 1100 = 0000

$C_2 = \mathbf{0000}$

Block 3

1111 XOR 0000 = 1111

1111 XOR 1100 = 0011

$C_3 = 0011$

Block 4

$1010 \text{ XOR } 0011 = 1001$

$1001 \text{ XOR } 1100 = 0101$

$C_4 = 0101$

CBC Ciphertext

[0110, 0000, 0011, 0101]

b. Compare the ciphertext outputs

Mode Ciphertext

ECB [0110, 0110, 0011, 0110]

CBC [0110, 0000, 0011, 0101]

c. Which mode hides patterns better?

CBC mode hides patterns more effectively because each plaintext block is combined with the previous ciphertext block before encryption. Even if two plaintext blocks are identical, their ciphertexts become different. In contrast, ECB encrypts identical plaintext blocks into identical ciphertext blocks, making patterns visible. Therefore, CBC provides better security than ECB.

3. DES vs 3-DES vs Blowfish Performance Study

Given:

- DES = 2 ms/block
- 3-DES = 6 ms/block
- Blowfish = 1 ms/block
- Number of blocks = 1000

i. Total encryption time

DES

$$1000 \times 2 = 2000 \text{ ms}$$

= 2 seconds

3-DES

$$1000 \times 6 = 6000 \text{ ms}$$

= 6 seconds

Blowfish

$$1000 \times 1 = 1000 \text{ ms}$$

= 1 second

ii. Security vs Performance

Algorithm	Key Size	Time	Analysis
DES	56 bits	2 s	Fast but weak because the key size is too small.
3-DES	168 bits	6 s	Much more secure than DES but significantly slower.
Blowfish	128 bits	1 s	Provides strong security with the fastest encryption speed among the three.

iii. Best algorithm for real-time communication

Recommendation: Blowfish

Reason:

- Fastest encryption (1 second for 1000 blocks)
- Strong 128-bit key
- Lower processing overhead
- Suitable for applications where both speed and security are important, such as real-time communication.

4. Diffie–Hellman Key Exchange Analysis

Given:

- $p = 23$
- $g = 5$
- A's private key = 6
- B's private key = 15

d. Compute public keys

Formula:

$$Public = g^{Private} p$$

User A

$$5^6 23$$
$$15625 23 = 8$$

A's Public Key = 8

User B

$$5^{15} 23$$
$$= 19$$

B's Public Key = 19

e. Compute the shared secret

User A

$$19^6 23 = 2$$

User B

$$8^{15} 23 = 2$$

Shared Secret Key = 2

f. Man-in-the-Middle (MITM) attack

In a Man-in-the-Middle attack, an attacker intercepts the public keys exchanged between User A and User B. Instead of forwarding the original keys, the attacker sends fake public keys to both users. As a result, User A and the attacker establish one shared key, while User B and the attacker establish another. The attacker can decrypt, read, modify, and re-encrypt messages without either user realizing the communication has been compromised.

g. How to prevent the attack

The most effective way to prevent a Man-in-the-Middle attack is to authenticate the exchanged public keys using **digital certificates** and **digital signatures**. Protocols such as TLS/SSL verify the identity of the communicating parties before the key exchange, ensuring that the public keys are genuine and have not been altered by an attacker.

5. ECC vs RSA Comparison

Given:

- RSA key size = **1024 bits**
- ECC key size = **160 bits**
- IoT device with limited processing power

a. RSA Key Size = 1024 bits

RSA uses a **1024-bit key** to provide security through the mathematical difficulty of factoring large prime numbers. Although RSA-1024 was considered secure in the past, it is now regarded as less secure than modern standards. RSA also requires larger keys, which increases computation time and memory usage.

b. ECC Key Size = 160 bits (Equivalent Security)

Elliptic Curve Cryptography (ECC) uses a **160-bit key** to provide approximately the same security level as a **1024-bit RSA key**. ECC is based on the Elliptic Curve Discrete Logarithm Problem, which is much harder to solve than the mathematical problem used in RSA. Therefore, ECC achieves the same security with much smaller keys.

c. Device Processing Capacity = Limited (IoT Device)

IoT devices generally have:

- Low processing power
- Limited memory (RAM)
- Limited battery life
- Low storage capacity

Because of these limitations, cryptographic algorithms used in IoT devices must consume fewer system resources while still providing strong security.

d. Analyze Computational Cost

RSA requires larger keys and more complex computations, resulting in higher processing time. ECC uses smaller keys and performs fewer calculations, making it much faster and more efficient on devices with limited hardware resources.

Result: ECC has a lower computational cost than RSA.

e. Estimate Which Consumes Less Power and Memory

ECC consumes less power because it performs fewer mathematical operations. It also requires less memory to store keys and execute cryptographic operations. RSA consumes more battery power and memory due to its larger key size and higher computational requirements.

Result: ECC consumes less power and memory than RSA.

f. Justify Which Cryptosystem is Better

For an IoT device with limited processing capacity, **ECC is the better choice** because:

- It provides the same security as RSA with much smaller keys.
- It requires less processing power.
- It uses less memory and storage.
- It consumes less battery power.
- It performs encryption and decryption faster on resource-constrained devices.

Conclusion: ECC is the most suitable cryptographic system for IoT devices because it offers strong security while minimizing computational cost, memory usage, and energy consumption.